ASSIGNMENT No: 08 Title: Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using the standard Heart Disease Data Set (You can use Java/Python ML library classes/API.

Problem Statement: To construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using the standard Heart Disease Data Set (You can use Java/Python ML library classes/API.

Aditya Rade Roll.No: BEAD23167

In [ ]:
```
%pip install pgmpy pandas numpy
```

In [ ]:
```
!pip install --upgrade jupyter ipywidgets
```

In [ ]:
```

```

In [4]:
```
# Read Cleveland Heart Disease dataset
heartDisease = pd.read_csv("heart.csv")

# Replace '?' with NaN
heartDisease = heartDisease.replace('?', np.nan)

# Display first five records
print("Few examples from the dataset:")
print(heartDisease.head())

print("\nDataset Shape:")
print(heartDisease.shape)
```

```
Few examples from the dataset:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0

Dataset Shape:
(1025, 14)
```

In [10]:
```
# Convert columns to numeric where possible
for column in heartDisease.columns:
    heartDisease[column] = pd.to_numeric(
        heartDisease[column], errors='coerce'
    )
```

```
# Remove rows containing missing values
heartDisease = heartDisease.dropna()

print("Dataset after preprocessing:")
print(heartDisease.head())

print("\nShape after preprocessing:")
print(heartDisease.shape)
```

```
Dataset after preprocessing:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0

Shape after preprocessing:
(1025, 14)
```

In [11]:
```
model = DiscreteBayesianNetwork([
    ('age', 'trestbps'),
    ('age', 'fbs'),
    ('sex', 'trestbps'),
    ('exang', 'trestbps'),
    ('trestbps', 'heartdisease'),
    ('fbs', 'heartdisease'),
    ('heartdisease', 'restecg'),
    ('heartdisease', 'thalach'),
    ('heartdisease', 'chol')
])

print("Bayesian Network Structure:")
print(model.edges())
```

```
Bayesian Network Structure:
[('age', 'trestbps'), ('age', 'fbs'), ('trestbps', 'heartdisease'), ('fbs', 'heartdi
sease'), ('sex', 'trestbps'), ('exang', 'trestbps'), ('heartdisease', 'restecg'),
('heartdisease', 'thalach'), ('heartdisease', 'chol')]
```

In [15]:
```
heartDisease.columns = heartDisease.columns.str.strip()

print("Columns before:", heartDisease.columns.tolist())

# Rename the last column to heartdisease
heartDisease = heartDisease.rename(
    columns={heartDisease.columns[-1]: 'heartdisease'}
)
```

```
print("Columns after:", heartDisease.columns.tolist())
```

```
Columns before: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalac
h', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
Columns after: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
'exang', 'oldpeak', 'slope', 'ca', 'thal', 'heartdisease']
```

In [17]:
```
from pgmpy.estimators import MaximumLikelihoodEstimator

print("\nLearning CPDs using Maximum Likelihood Estimation...")

estimator = MaximumLikelihoodEstimator(model, heartDisease)

cpds = estimator.get_parameters()

for cpd in cpds:
    model.add_cpds(cpd)

print("\nCPDs learned successfully!")
print("Model check:", model.check_model())
```

```
Learning CPDs using Maximum Likelihood Estimation...
C:\Users\adity\AppData\Local\Temp\ipykernel_19708\2819115343.py:5: FutureWarning: `p
gmpy.estimators.MaximumLikelihoodEstimator` is deprecated and will be removed in v1.
3.0. Please use `pgmpy.parameter_estimator.DiscreteMLE` instead.
  estimator = MaximumLikelihoodEstimator(model, heartDisease)
CPDs learned successfully!
Model check: True
```

In [18]:
```
heartDisease = heartDisease.replace('?', np.nan)
heartDisease = heartDisease.dropna()
```

In [19]:
```
import numpy as np
import pandas as pd

heartDisease = pd.read_csv("heart.csv")

# Clean column names
heartDisease.columns = heartDisease.columns.str.strip()

# Replace missing values
heartDisease = heartDisease.replace('?', np.nan)
heartDisease = heartDisease.dropna()

# Make sure diagnosis column is named correctly
if 'heartdisease' not in heartDisease.columns:
    heartDisease = heartDisease.rename(
        columns={heartDisease.columns[-1]: 'heartdisease'}
    )

print("Final columns:")
print(heartDisease.columns.tolist())

print("\nDataset shape:", heartDisease.shape)
```

```
Final columns:
['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldp
eak', 'slope', 'ca', 'thal', 'heartdisease']

Dataset shape: (1025, 14)
```

In [20]:
```
print(heartDisease.columns.tolist())
```

```
['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldp
eak', 'slope', 'ca', 'thal', 'heartdisease']
```

In [21]:
```
import numpy as np
import pandas as pd

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator

# Load dataset
heartDisease = pd.read_csv("heart.csv")

# Clean column names
heartDisease.columns = heartDisease.columns.str.strip()

# Handle missing values
heartDisease = heartDisease.replace("?", np.nan)
heartDisease = heartDisease.dropna()

# Rename last column as heartdisease
if "heartdisease" not in heartDisease.columns:
    heartDisease = heartDisease.rename(
        columns={heartDisease.columns[-1]: "heartdisease"}
    )

print("Dataset columns:")
print(heartDisease.columns.tolist())

# Create Bayesian Network
model = DiscreteBayesianNetwork([
    ("age", "trestbps"),
    ("age", "fbs"),
    ("sex", "trestbps"),
    ("exang", "trestbps"),
    ("trestbps", "heartdisease"),
    ("fbs", "heartdisease"),
    ("heartdisease", "restecg"),
    ("heartdisease", "thalach"),
    ("heartdisease", "chol")
])

# Learn CPDs
estimator = MaximumLikelihoodEstimator(model, heartDisease)
cpds = estimator.get_parameters()

for cpd in cpds:
    model.add_cpds(cpd)
```

```
print("\nBayesian Network created successfully.")
print("Model check:", model.check_model())

print("\nLearned CPDs:")
for cpd in model.get_cpds():
    print(cpd)
```

```
Dataset columns:
['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldp
eak', 'slope', 'ca', 'thal', 'heartdisease']
```

```
C:\Users\adity\AppData\Local\Temp\ipykernel_19708\3823999470.py:40: FutureWarning: `
pgmpy.estimators.MaximumLikelihoodEstimator` is deprecated and will be removed in v
1.3.0. Please use `pgmpy.parameter_estimator.DiscreteMLE` instead.
  estimator = MaximumLikelihoodEstimator(model, heartDisease)
```

```
Bayesian Network created successfully.
Model check: True

Learned CPDs:
+---------+------------+
| age(29) | 0.00390244 |
+---------+------------+
| age(34) | 0.00585366 |
+---------+------------+
| age(35) | 0.0146341  |
+---------+------------+
| age(37) | 0.00585366 |
+---------+------------+
| age(38) | 0.0117073  |
+---------+------------+
| age(39) | 0.0136585  |
+---------+------------+
| age(40) | 0.0107317  |
+---------+------------+
| age(41) | 0.0312195  |
+---------+------------+
| age(42) | 0.0253659  |
+---------+------------+
| age(43) | 0.0253659  |
+---------+------------+
| age(44) | 0.035122   |
+---------+------------+
| age(45) | 0.0243902  |
+---------+------------+
| age(46) | 0.022439   |
+---------+------------+
| age(47) | 0.017561   |
+---------+------------+
| age(48) | 0.022439   |
+---------+------------+
| age(49) | 0.0165854  |
+---------+------------+
| age(50) | 0.0204878  |
+---------+------------+
| age(51) | 0.0380488  |
+---------+------------+
| age(52) | 0.0419512  |
+---------+------------+
| age(53) | 0.0253659  |
+---------+------------+
| age(54) | 0.0517073  |
+---------+------------+
| age(55) | 0.0292683  |
+---------+------------+
| age(56) | 0.0380488  |
+---------+------------+
| age(57) | 0.0556098  |
+---------+------------+
| age(58) | 0.0663415  |
+---------+------------+
| age(59) | 0.044878   |
```

age(60)	0.0360976
age(61)	0.0302439
age(62)	0.0360976
age(63)	0.0312195
age(64)	0.0331707
age(65)	0.0263415
age(66)	0.0243902
age(67)	0.0302439
age(68)	0.0117073
age(69)	0.00878049
age(70)	0.0136585
age(71)	0.0107317
age(74)	0.00292683
age(76)	0.00292683
age(77)	0.00292683

age	...	age(77)	age(77)
exang	...	exang(1)	exang(1)
sex	...	sex(0)	sex(1)
trestbps(94)	...	0.02040816326530612	0.0
trestbps(100)	...	0.02040816326530612	0.0
trestbps(101)	...	0.02040816326530612	0.0
trestbps(102)	...	0.02040816326530612	0.0
trestbps(104)	...	0.02040816326530612	0.0
trestbps(105)	...	0.02040816326530612	0.0
trestbps(106)	...	0.02040816326530612	0.0
trestbps(108)	...	0.02040816326530612	0.0
trestbps(110)	...	0.02040816326530612	0.0

trestbps(112)	...	0.02040816326530612	0.0
trestbps(114)	...	0.02040816326530612	0.0
trestbps(115)	...	0.02040816326530612	0.0
trestbps(117)	...	0.02040816326530612	0.0
trestbps(118)	...	0.02040816326530612	0.0
trestbps(120)	...	0.02040816326530612	0.0
trestbps(122)	...	0.02040816326530612	0.0
trestbps(123)	...	0.02040816326530612	0.0
trestbps(124)	...	0.02040816326530612	0.0
trestbps(125)	...	0.02040816326530612	1.0
trestbps(126)	...	0.02040816326530612	0.0
trestbps(128)	...	0.02040816326530612	0.0
trestbps(129)	...	0.02040816326530612	0.0
trestbps(130)	...	0.02040816326530612	0.0
trestbps(132)	...	0.02040816326530612	0.0
trestbps(134)	...	0.02040816326530612	0.0
trestbps(135)	...	0.02040816326530612	0.0
trestbps(136)	...	0.02040816326530612	0.0
trestbps(138)	...	0.02040816326530612	0.0
trestbps(140)	...	0.02040816326530612	0.0
trestbps(142)	...	0.02040816326530612	0.0
trestbps(144)	...	0.02040816326530612	0.0
trestbps(145)	...	0.02040816326530612	0.0
trestbps(146)	...	0.02040816326530612	0.0
trestbps(148)	...	0.02040816326530612	0.0
trestbps(150)	...	0.02040816326530612	0.0
trestbps(152)	...	0.02040816326530612	0.0
trestbps(154)	...	0.02040816326530612	0.0

trestbps(155)	...	0.02040816326530612	0.0
trestbps(156)	...	0.02040816326530612	0.0
trestbps(160)	...	0.02040816326530612	0.0
trestbps(164)	...	0.02040816326530612	0.0
trestbps(165)	...	0.02040816326530612	0.0
trestbps(170)	...	0.02040816326530612	0.0
trestbps(172)	...	0.02040816326530612	0.0
trestbps(174)	...	0.02040816326530612	0.0
trestbps(178)	...	0.02040816326530612	0.0
trestbps(180)	...	0.02040816326530612	0.0
trestbps(192)	...	0.02040816326530612	0.0
trestbps(200)	...	0.02040816326530612	0.0

age	age(29)	age(34)	...	age(74)	age(76)	age(77)
fbs(0)	1.0	1.0	...	1.0	1.0	1.0
fbs(1)	0.0	0.0	...	0.0	0.0	0.0

sex(0)	0.30439
sex(1)	0.69561

exang(0)	0.663415
exang(1)	0.336585

fbs	fbs(0)	...	fbs(1)	fbs(1)
trestbps	trestbps(94)	...	trestbps(192)	trestbps(200)
heartdisease(0)	0.0	...	0.5	1.0
heartdisease(1)	1.0	...	0.5	0.0

heartdisease	heartdisease(0)	heartdisease(1)
restecg(0)	0.5671342685370742	0.4068441064638783
restecg(1)	0.4088176352705411	0.5874524714828897

restecg(2)	0.02404809619238477	0.005703422053231939

heartdisease	heartdisease(0)	heartdisease(1)
thalach(71)	0.008016032064128256	0.0
thalach(88)	0.006012024048096192	0.0
thalach(90)	0.006012024048096192	0.0
thalach(95)	0.008016032064128256	0.0
thalach(96)	0.008016032064128256	0.005703422053231939
thalach(97)	0.008016032064128256	0.0
thalach(99)	0.006012024048096192	0.0
thalach(103)	0.01603206412825651	0.0
thalach(105)	0.014028056112224449	0.005703422053231939
thalach(106)	0.006012024048096192	0.0
thalach(108)	0.01603206412825651	0.0
thalach(109)	0.014028056112224449	0.0
thalach(111)	0.014028056112224449	0.005703422053231939
thalach(112)	0.014028056112224449	0.0
thalach(113)	0.006012024048096192	0.0
thalach(114)	0.01603206412825651	0.005703422053231939
thalach(115)	0.006012024048096192	0.011406844106463879
thalach(116)	0.008016032064128256	0.005703422053231939
thalach(117)	0.008016032064128256	0.0
thalach(118)	0.008016032064128256	0.0
thalach(120)	0.022044088176352707	0.0
thalach(121)	0.0	0.005703422053231939
thalach(122)	0.006012024048096192	0.017110266159695818
thalach(123)	0.006012024048096192	0.005703422053231939
thalach(124)	0.008016032064128256	0.0

thalach(125)	0.036072144288577156	0.013307984790874524
thalach(126)	0.022044088176352707	0.005703422053231939
thalach(127)	0.008016032064128256	0.0
thalach(128)	0.006012024048096192	0.0
thalach(129)	0.008016032064128256	0.0
thalach(130)	0.022044088176352707	0.0076045627376425855
thalach(131)	0.014028056112224449	0.013307984790874524
thalach(132)	0.04408817635270541	0.0076045627376425855
thalach(133)	0.008016032064128256	0.005703422053231939
thalach(134)	0.008016032064128256	0.0
thalach(136)	0.014028056112224449	0.0
thalach(137)	0.0	0.005703422053231939
thalach(138)	0.008016032064128256	0.013307984790874524
thalach(139)	0.008016032064128256	0.005703422053231939
thalach(140)	0.03006012024048096	0.011406844106463879
thalach(141)	0.02004008016032064	0.0
thalach(142)	0.026052104208416832	0.011406844106463879
thalach(143)	0.02004008016032064	0.024714828897338403
thalach(144)	0.03807615230460922	0.013307984790874524
thalach(145)	0.022044088176352707	0.005703422053231939
thalach(146)	0.022044088176352707	0.005703422053231939
thalach(147)	0.02004008016032064	0.013307984790874524
thalach(148)	0.0	0.017110266159695818
thalach(149)	0.0	0.011406844106463879
thalach(150)	0.036072144288577156	0.013307984790874524
thalach(151)	0.0	0.022813688212927757
thalach(152)	0.01603206412825651	0.03802281368821293
thalach(153)	0.006012024048096192	0.011406844106463879

thalach(154)	0.014028056112224449	0.019011406844106463
thalach(155)	0.014028056112224449	0.013307984790874524
thalach(156)	0.022044088176352707	0.019011406844106463
thalach(157)	0.006012024048096192	0.022813688212927757
thalach(158)	0.02004008016032064	0.017110266159695818
thalach(159)	0.008016032064128256	0.017110266159695818
thalach(160)	0.03206412825651302	0.028517110266159697
thalach(161)	0.014028056112224449	0.017110266159695818
thalach(162)	0.022044088176352707	0.045627376425855515
thalach(163)	0.012024048096192385	0.043726235741444866
thalach(164)	0.008016032064128256	0.005703422053231939
thalach(165)	0.014028056112224449	0.019011406844106463
thalach(166)	0.006012024048096192	0.013307984790874524
thalach(167)	0.0	0.005703422053231939
thalach(168)	0.01603206412825651	0.017110266159695818
thalach(169)	0.01603206412825651	0.026615969581749048
thalach(170)	0.008016032064128256	0.024714828897338403
thalach(171)	0.008016032064128256	0.019011406844106463
thalach(172)	0.0	0.039923954372623575
thalach(173)	0.01603206412825651	0.03802281368821293
thalach(174)	0.014028056112224449	0.019011406844106463
thalach(175)	0.0	0.019011406844106463
thalach(177)	0.006012024048096192	0.0
thalach(178)	0.0	0.028517110266159697
thalach(179)	0.0	0.030418250950570342
thalach(180)	0.0	0.011406844106463879
thalach(181)	0.008016032064128256	0.005703422053231939
thalach(182)	0.008016032064128256	0.026615969581749048

thalach(184)	0.0	0.005703422053231939
thalach(185)	0.0	0.005703422053231939
thalach(186)	0.0	0.011406844106463879
thalach(187)	0.0	0.005703422053231939
thalach(188)	0.0	0.005703422053231939
thalach(190)	0.0	0.0076045627376425855
thalach(192)	0.0	0.005703422053231939
thalach(194)	0.0	0.005703422053231939
thalach(195)	0.006012024048096192	0.0
thalach(202)	0.0	0.0076045627376425855

heartdisease	heartdisease(0)	heartdisease(1)
chol(126)	0.0	0.005703422053231939
chol(131)	0.006012024048096192	0.0
chol(141)	0.0	0.005703422053231939
chol(149)	0.008016032064128256	0.0076045627376425855
chol(157)	0.0	0.0076045627376425855
chol(160)	0.0	0.005703422053231939
chol(164)	0.006012024048096192	0.0
chol(166)	0.008016032064128256	0.0
chol(167)	0.008016032064128256	0.0
chol(168)	0.0	0.005703422053231939
chol(169)	0.008016032064128256	0.0
chol(172)	0.006012024048096192	0.0
chol(174)	0.008016032064128256	0.0
chol(175)	0.0	0.02091254752851711
chol(176)	0.006012024048096192	0.0
chol(177)	0.01603206412825651	0.011406844106463879
chol(178)	0.0	0.005703422053231939

chol(180)	0.0	0.0076045627376425855
chol(182)	0.0	0.005703422053231939
chol(183)	0.0	0.0076045627376425855
chol(184)	0.006012024048096192	0.0
chol(185)	0.006012024048096192	0.0
chol(186)	0.0	0.0076045627376425855
chol(187)	0.008016032064128256	0.0
chol(188)	0.014028056112224449	0.0
chol(192)	0.0	0.013307984790874524
chol(193)	0.006012024048096192	0.005703422053231939
chol(195)	0.0	0.005703422053231939
chol(196)	0.0	0.011406844106463879
chol(197)	0.012024048096192385	0.024714828897338403
chol(198)	0.008016032064128256	0.005703422053231939
chol(199)	0.0	0.017110266159695818
chol(200)	0.006012024048096192	0.0
chol(201)	0.0	0.017110266159695818
chol(203)	0.01603206412825651	0.0076045627376425855
chol(204)	0.01603206412825651	0.024714828897338403
chol(205)	0.008016032064128256	0.005703422053231939
chol(206)	0.01603206412825651	0.0
chol(207)	0.008016032064128256	0.005703422053231939
chol(208)	0.0	0.011406844106463879
chol(209)	0.0	0.013307984790874524
chol(210)	0.0	0.005703422053231939
chol(211)	0.0	0.024714828897338403
chol(212)	0.03006012024048096	0.005703422053231939
chol(213)	0.0	0.011406844106463879

chol(214)	0.0	0.011406844106463879
chol(215)	0.0	0.005703422053231939
chol(216)	0.006012024048096192	0.005703422053231939
chol(217)	0.008016032064128256	0.0
chol(218)	0.01603206412825651	0.0
chol(219)	0.008016032064128256	0.011406844106463879
chol(220)	0.0	0.022813688212927757
chol(221)	0.0	0.013307984790874524
chol(222)	0.0	0.013307984790874524
chol(223)	0.008016032064128256	0.011406844106463879
chol(224)	0.008016032064128256	0.0
chol(225)	0.01603206412825651	0.0
chol(226)	0.0	0.024714828897338403
chol(227)	0.0	0.015209125475285171
chol(228)	0.008016032064128256	0.0076045627376425855
chol(229)	0.02404809619238477	0.0
chol(230)	0.022044088176352707	0.0
chol(231)	0.014028056112224449	0.005703422053231939
chol(232)	0.008016032064128256	0.005703422053231939
chol(233)	0.006012024048096192	0.017110266159695818
chol(234)	0.01603206412825651	0.024714828897338403
chol(235)	0.0	0.011406844106463879
chol(236)	0.006012024048096192	0.011406844106463879
chol(237)	0.008016032064128256	0.0
chol(239)	0.014028056112224449	0.011406844106463879
chol(240)	0.0	0.026615969581749048
chol(241)	0.006012024048096192	0.0
chol(242)	0.0	0.005703422053231939

chol(243)	0.014028056112224449	0.011406844106463879
chol(244)	0.006012024048096192	0.011406844106463879
chol(245)	0.0	0.017110266159695818
chol(246)	0.014028056112224449	0.005703422053231939
chol(247)	0.006012024048096192	0.005703422053231939
chol(248)	0.006012024048096192	0.005703422053231939
chol(249)	0.022044088176352707	0.0
chol(250)	0.0	0.017110266159695818
chol(252)	0.0	0.005703422053231939
chol(253)	0.006012024048096192	0.0076045627376425855
chol(254)	0.028056112224448898	0.005703422053231939
chol(255)	0.006012024048096192	0.005703422053231939
chol(256)	0.01603206412825651	0.005703422053231939
chol(257)	0.0	0.005703422053231939
chol(258)	0.012024048096192385	0.0076045627376425855
chol(259)	0.006012024048096192	0.0
chol(260)	0.008016032064128256	0.005703422053231939
chol(261)	0.008016032064128256	0.005703422053231939
chol(262)	0.0	0.005703422053231939
chol(263)	0.008016032064128256	0.011406844106463879
chol(264)	0.006012024048096192	0.005703422053231939
chol(265)	0.0	0.013307984790874524
chol(266)	0.006012024048096192	0.005703422053231939
chol(267)	0.006012024048096192	0.005703422053231939
chol(268)	0.008016032064128256	0.005703422053231939
chol(269)	0.014028056112224449	0.017110266159695818
chol(270)	0.006012024048096192	0.005703422053231939
chol(271)	0.0	0.011406844106463879

chol(273)	0.006012024048096192	0.005703422053231939
chol(274)	0.018036072144288578	0.0
chol(275)	0.008016032064128256	0.005703422053231939
chol(276)	0.008016032064128256	0.0
chol(277)	0.0	0.011406844106463879
chol(278)	0.0	0.0076045627376425855
chol(281)	0.008016032064128256	0.0
chol(282)	0.028056112224448898	0.0
chol(283)	0.014028056112224449	0.005703422053231939
chol(284)	0.008016032064128256	0.0
chol(286)	0.01603206412825651	0.0
chol(288)	0.01603206412825651	0.005703422053231939
chol(289)	0.01603206412825651	0.0
chol(290)	0.006012024048096192	0.0
chol(293)	0.008016032064128256	0.0
chol(294)	0.006012024048096192	0.005703422053231939
chol(295)	0.0	0.011406844106463879
chol(298)	0.006012024048096192	0.005703422053231939
chol(299)	0.014028056112224449	0.0
chol(300)	0.008016032064128256	0.0
chol(302)	0.0	0.011406844106463879
chol(303)	0.0	0.017110266159695818
chol(304)	0.006012024048096192	0.005703422053231939
chol(305)	0.006012024048096192	0.0
chol(306)	0.0	0.005703422053231939
chol(307)	0.008016032064128256	0.0
chol(308)	0.0	0.011406844106463879
chol(309)	0.014028056112224449	0.0076045627376425855

```
+--------------+----------------------+----------------------+
| chol(311)    | 0.008016032064128256 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(313)    | 0.0                  | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(315)    | 0.008016032064128256 | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(318)    | 0.008016032064128256 | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(319)    | 0.008016032064128256 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(321)    | 0.0                  | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(322)    | 0.008016032064128256 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(325)    | 0.0                  | 0.011406844106463879 |
+--------------+----------------------+----------------------+
| chol(326)    | 0.006012024048096192 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(327)    | 0.008016032064128256 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(330)    | 0.01603206412825651  | 0.0                  |
+--------------+----------------------+----------------------+
| chol(335)    | 0.01603206412825651  | 0.0                  |
+--------------+----------------------+----------------------+
| chol(340)    | 0.0                  | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(341)    | 0.008016032064128256 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(342)    | 0.0                  | 0.0076045627376425855 |
+--------------+----------------------+----------------------+
| chol(353)    | 0.008016032064128256 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(354)    | 0.0                  | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(360)    | 0.0                  | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(394)    | 0.0                  | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(407)    | 0.008016032064128256 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(409)    | 0.006012024048096192 | 0.0                  |
+--------------+----------------------+----------------------+
| chol(417)    | 0.0                  | 0.005703422053231939 |
+--------------+----------------------+----------------------+
| chol(564)    | 0.0                  | 0.005703422053231939 |
+--------------+----------------------+----------------------+
```

In [24]:
```
from pgmpy.inference import VariableElimination

# Create inference object
heartDisease_infer = VariableElimination(model)

# Diagnosis using age = 63
print("\n--- Diagnosis using Age = 63 ---")
```

```
q1 = heartDisease_infer.query(
    variables=["heartdisease"],
    evidence={"age": 63}
)

print(q1)


# Diagnosis using cholesterol = 233
print("\n--- Diagnosis using Cholesterol = 233 ---")

q2 = heartDisease_infer.query(
    variables=["heartdisease"],
    evidence={"chol": 233}
)

print(q2)
print("Aditya Rade   BEAD23167")
```

```
--- Diagnosis using Age = 63 ---
+-----------------+---------------------+
| heartdisease    |   phi(heartdisease) |
+=================+=====================+
| heartdisease(0) |              0.4945 |
+-----------------+---------------------+
| heartdisease(1) |              0.5055 |
+-----------------+---------------------+

--- Diagnosis using Cholesterol = 233 ---
+-----------------+---------------------+
| heartdisease    |   phi(heartdisease) |
+=================+=====================+
| heartdisease(0) |              0.2432 |
+-----------------+---------------------+
| heartdisease(1) |              0.7568 |
+-----------------+---------------------+
Aditya Rade   BEAD23167
```